Modules: ESM vs CommonJS

A module is a file whose top-level variables are private by default and whose exports are the only things other files can see. JavaScript shipped without any module system, so the ecosystem invented two: **CommonJS** (CJS) — Node's original `require` / `module.exports` — and **ES Modules** (ESM) — the `import` / `export` syntax standardized in ES2015 and now native to both browsers and Node. Interviewers probe this because the two look interchangeable but differ in loading (sync vs async), binding semantics (value copy vs live binding), and static analyzability (tree-shaking), and because mixing them is a real source of production bugs.

Why modules at all

Before modules, code lived in `<script>` tags that all shared one global object. Two files could clobber each other's variables, load order was manual and fragile, and there was no way to state "this file needs that file" — dependencies were implicit. The community's stopgap was the **IIFE** (Immediately Invoked Function Expression): wrap a file in a function so its internals stay private, and hang only the public API on `window`.

JS Copy

```
// The pre-module pattern: an IIFE for scope isolation
const Counter = (function () {
  let count = 0; // private — not visible outside
  return {
    increment() { count++; },
    get() { return count; },
  };
})();
Counter.increment();
```

Modules make this a language feature: **scope isolation** is automatic (top-level `const` / `let` / `function` are file-local, not global), and **dependencies are explicit** (`import` / `require` names exactly what a file needs, so tooling can build the dependency graph, order evaluation, and bundle correctly).

ES Modules (ESM)

ESM is the standard. You export bindings and import them by name.

JS Copy

```
// math.mjs
export const PI = 3.14159;          // named export
export function square(x) {         // named export
  return x * x;
}
export default function cube(x) {   // the single default export
  return x * x * x;
}
```

JS Copy

```
// app.mjs
import cube, { PI, square } from './math.mjs'; // default + named in one l
import * as math from './math.mjs';            // namespace object
console.log(square(4), cube(3), PI); // ⇒ 16 27 3.14159
console.log(math.PI);                // ⇒ 3.14159
```

Key properties, each of which interviewers ask about:

- **Static structure.** `import` / `export` are declarations, not function calls. They must sit at the top level (not inside an `if` or a function) and their specifiers must be string literals. This lets tools resolve the whole graph *without running the code*.

- **Hoisted imports.** All imports are resolved and the imported modules evaluated *before* the importing module's body runs — regardless of where the `import` line physically appears. Import is like a hoisted declaration.
- **Live bindings.** An import is a read-only *view* onto the exporting module's variable, not a snapshot. If the exporter reassigns it later, importers see the new value.

JS Copy

```
// counter.mjs
export let count = 0;
export function increment() { count++; }
```

JS Copy

```
// main.mjs
import { count, increment } from './counter.mjs';
console.log(count); // ⇒ 0
increment();
console.log(count); // ⇒ 1  (live binding: reflects the exporter's update
```

- **Always strict mode.** Every ESM is implicitly in strict mode — no `with`, assigning to an undeclared variable throws, etc. There is no `"use strict"` needed.
- `this` is `undefined` at the top level (not the global object).
- **Deferred, like `<script defer>`.** In the browser, `<script type="module">` is deferred by default: it downloads in parallel but executes after HTML parsing, in order.
- **Dynamic `import()`.** When you need conditional or lazy loading, `import()` is a function-like form that returns a **Promise** of the module namespace. Unlike static `import`, it can appear anywhere and take a computed specifier — this is what enables code-splitting.

JS Copy

```
// Load a module on demand; import() resolves to the namespace object
async function loadOs() {
  const os = await import('node:os');
  return typeof os.platform; // ⇒ "function"
```

In Node, ESM is selected by a `.mjs` extension or `"type": "module"` in `package.json`.

CommonJS (CJS)

CommonJS predates ESM by years — it's what Node shipped with in 2009, and npm grew up around it. A file uses `require()` to pull in dependencies and assigns to `module.exports` (or the `exports` alias) to expose its API.

JS Copy

```
// math.cjs
function square(x) { return x * x; }
const PI = 3.14159;
module.exports = { square, PI }; // whatever you assign becomes the module
```

JS Copy

```
// app.cjs
const { square, PI } = require('./math.cjs'); // require returns module.ex
console.log(square(5), PI); // ⇒ 25 3.14159
```

Its defining traits:

- **Synchronous loading.** `require()` reads, compiles, and runs the target file *right now*, blocking until it returns. This was fine on a server with local files; it's why CJS never worked natively in browsers.
- **Dynamic and imperative.** `require()` is a real function call. You can call it conditionally, in a loop, with a computed path, and the return value is a plain object you can inspect at runtime. Powerful, but opaque to static tooling.
- **Value-copy semantics.** `require` gives you whatever the module *assigned* to `module.exports`. Primitive exports are effectively snapshots — later reassignments

in the exporter are not seen (contrast ESM's live bindings).

JS Copy

```
// counter.cjs
let count = 0;
function increment() { count++; }
module.exports = { count, increment }; // count's value (0) is copied onto
```

JS Copy

```
// main.cjs
const m = require('./counter.cjs');
console.log(m.count); // ⇒ 0
m.increment();
console.log(m.count); // ⇒ 0  (still 0 — the exported number was copied,
```

- `module.exports` **vs** `exports`. `exports` is just a local variable initialized to point at `module.exports`. Mutating it (`exports.foo = 1`) works because you're touching the shared object. **Reassigning** it (`exports = { ... }`) silently breaks the link and exports nothing — a classic trap (see Gotchas).
- **Module-scope helpers.** CJS modules run inside a wrapper function that provides `require`, `module`, `exports`, `__dirname` (the directory of the current file), and `__filename`. Those last two do **not** exist in ESM.
- `this` **at top level is** `module.exports`, not `undefined`.

ESM vs CommonJS: side-by-side

Aspect	ES Modules (ESM)	CommonJS (CJS)
Syntax	`import` / `export` (declarations)	`require()` / `module.exports` (func
Loading	Asynchronous, parallelizable	Synchronous, blocking
Structure	Static — resolvable without running	Dynamic — resolved at runtime

Placement	Top level only (statically)	Anywhere (`require` is a call)
Binding semantics	**Live bindings** (read-only views)	**Value copy** at export time
Tree-shaking	Yes (static graph)	No (dynamic, opaque)
Top-level `await`	Supported	Not supported
Strict mode	Always on	Off unless `"use strict"`
`this` at top level	`undefined`	`module.exports`
`__dirname` / `__filename`	Absent (use `import.meta.url`)	Present
Dynamic loading	`import()` → Promise	`require()` → value (sync)
Native environment	Browsers + Node	Node (via bundlers in browser)

Interop and pitfalls

Calling CJS from ESM. Node lets an ESM file `import` a CommonJS module. The module's `module.exports` object becomes the ESM **default** export, and Node also tries to expose its keys as named exports (via static analysis — which can miss dynamically-assigned keys). So the reliable form is:

JS Copy

```
// import a CommonJS package from ESM
import pkg from 'some-cjs-lib';       // pkg === that module's module.expo
```

Calling ESM from CJS. The reverse is *not* synchronous: because ESM loads asynchronously, a CJS file cannot `require()` a real `.mjs` module — it must use dynamic `await import('./x.mjs')`, which returns a Promise.

The `__esModule` / default-interop mess. When bundlers (Babel, TypeScript, webpack) down-compile ESM to CJS, `export default x` becomes `module.exports.default = x` and a marker `module.exports.__esModule = true` is set. Interop helpers (often named `_interopRequireDefault` or `__esm`/`__toESM` in esbuild output) check that flag to decide whether to unwrap `.default`. When the flag is missing or the toolchains disagree, you get the infamous `import x from 'y'` **gives you** `{ default: [Function] }` **instead of the function**, or `TypeError: x is not a function`. The fix is usually matching your `esModuleInterop` / `allowSyntheticDefaultImports` settings to how the dependency was authored.

Tree-shaking is the concrete payoff of ESM's static structure. Because a bundler can see *exactly* which named exports each module imports — without executing anything — it can drop the exports nobody references (dead-code elimination). CJS defeats this: `require()` returns a runtime object and `module.exports` can be built up imperatively, so the bundler must conservatively keep the whole module. This is the main reason libraries ship ESM builds.

Circular dependencies — both systems handle A→B→A cycles by returning a *partial* module, but differently. In **CJS**, when B `require`s A mid-cycle, it gets A's `module.exports` *as it is so far* — often incomplete, so a field reads as `undefined`. In **ESM**, the bindings exist (hoisted) but may be in the **TDZ**: if B reads A's export at evaluation time before A initialized it, you get a `ReferenceError: Cannot access 'x' before initialization`. If instead B only reads it later, inside a function called after everything has loaded, the live binding is populated and it works. The practical takeaway: don't touch a circular import's value at module top level; defer it into a function.

JS Copy

```
// CJS circular: y.cjs requires x.cjs before x finished exporting
// x.cjs: const y = require('./y.cjs');  ...  module.exports = { done: true
```

```
// y.cjs: const x = require('./x.cjs'); console.log(x.done); module.export
// main: const y = require('./y.cjs'); console.log('y.done =', y.done);
//   y.cjs sees x.done = undefined   <-- partial exports
//   x.cjs sees y.done = true
```

Common interview questions

Q: What's the single biggest semantic difference between `import` and `require`?
Live bindings vs value copy. An ESM import is a live, read-only reference to the exporter's variable — reassigning it in the exporter updates every importer. A CJS `require` returns the value of `module.exports` at the moment it ran; reassignments after that aren't seen by earlier requirers.

Q: Why can't you `require()` an ES module in Node?
`require` is synchronous, but ESM resolution and evaluation are asynchronous (they can involve top-level `await` and async loading). You can't collapse an async load into a sync return, so Node requires dynamic `import()` (which returns a Promise) instead. (Recent Node versions permit `require()` of ESM only when the graph has no top-level `await`.)

Q: Why does ESM enable tree-shaking but CJS doesn't?
`import` / `export` are static — bindings are fixed string names known before execution — so a bundler can build the exact used/unused export graph and delete dead code. CJS's `require()` / `module.exports` are dynamic runtime values; the bundler can't prove an export is unused, so it keeps everything.

Q: What does `import * as ns from './m.js'` give you?
A **module namespace object** whose properties are live bindings to each named export (and `ns.default` for the default). It's sealed and its properties are read-only from the importer's side.

Q: Where can `import` statements appear, and where can `import()` appear?
Static `import` must be at the top level of a module — it can't be nested in a block or function, and its path must be a literal. Dynamic `import()` is expression-like: it works anywhere, accepts a computed specifier, and returns a Promise — the basis of lazy loading / code-splitting.

Q: Are ES modules in strict mode?
Always, implicitly. No directive needed. CommonJS is sloppy-mode unless you add `"use strict"`.

Gotchas / traps

Reassigning `exports` exports nothing. `exports` is a local alias for `module.exports`. Mutate it and you're fine; reassign it and you sever the link.

JS Copy

```
// bad.cjs
exports = { hi: 1 };   // reassigns the local var — module.exports is unto
// good.cjs
exports.hi = 1;        // mutates the shared object — works
// require('./bad.cjs')  ⇒ {}
// require('./good.cjs') ⇒ { hi: 1 }
```

`import` is hoisted — order in the file lies. Imported modules evaluate *before* any code in the importing file, even code physically above the `import`. You cannot run setup (e.g. set an env var) in the importing module before an imported module's top-level code sees it; the import already ran. Use dynamic `import()` if you truly need ordered side effects.

Live bindings are read-only for the importer. You can *observe* an exported `let` change, but you cannot assign to an import — `import { count } ...; count = 5;` is a `SyntaxError` (in ESM's strict, static world). Only the exporting module may mutate it.

`__dirname` is `undefined` in ESM. It doesn't exist. The ESM way is `import.meta.url`, e.g. `new URL('.', import.meta.url).pathname` or `fileURLToPath(import.meta.url)`. Copy-pasting CJS path code into a `.mjs` file breaks it.

Default-export interop surprises. Importing a CJS module that did `module.exports = fn` as `import fn from 'mod'` usually works in Node, but under some bundler/TS configs you receive `{ default: fn }` (or the reverse), yielding `x is not a function`. When it happens, it's an `esModuleInterop` / `__esModule` -flag mismatch, not your code.

object with `undefined` fields. Defer the access into a function that runs after load, and the live binding resolves correctly.

JS Copy

```
// ESM circular, safe form: read the import inside a function, not at top
// b.mjs: import { a } from './a.mjs'; export function getB(){ return 'see
// a.mjs: import { getB } from './b.mjs'; export const a = 'a'; console.lo
// prints: sees a   ← works, because getB() runs after a is initialized
```

QUICK CHECK — no marks, just for you.

1. This is the pre-module IIFE pattern for scope isolation. What does it log?

JAVASCRIPT Copy

```javascript
const Counter = (function () {
  let count = 0;            // private — not visible outside
  return {
    increment() { count++; },
    get() { return count; },
  };
})();
Counter.increment();
Counter.increment();
console.log(Counter.get(), typeof count);
```

- ○ 2 number
- ○ 1 undefined
- ○ undefined undefined
- ○ 2 undefined

2. What is the single biggest semantic difference between an ESM `import` and a CommonJS `require`?

- An ESM import is a live, read-only view onto the exporter's variable, so later reassignments are visible; `require` returns the value of `module.exports` at the moment it ran, so later reassignments are not seen.
- `require` is asynchronous and non-blocking, while `import` is synchronous and blocks until the module is evaluated.
- `import` copies each exported value at load time, while `require` sets up live bindings that update when the exporter changes.
- Both create live bindings, but only `import` can load a module more than once.

Check answer

3. Why does ESM enable tree-shaking (dead-code elimination) while CommonJS generally does not?

- ESM modules always run in strict mode, which lets bundlers safely delete unused code.
- `require()` executes asynchronously, so a bundler cannot pause long enough to analyze which exports are used.
- `import`/`export` are static declarations with fixed names known before execution, so a bundler can build the exact used/unused export graph without running the code; `require()`/`module.exports` are dynamic runtime values, so the bundler can't prove an export is unused and keeps everything.
- CommonJS modules are always wrapped in an IIFE, which hides their export names from the bundler.

Previous
Map, Set, WeakMap & WeakSet

Next
Symbols, Iterators & Proxy

LEARN MORE

ARTICLES & DOCS

- JavaScript modules (MDN)
  MDN
- ECMAScript modules (Node.js)
  nodejs.org

VIDEOS



- ESM vs CommonJS in Node
  YouTube



- Say goodbye to CommonJS, use ESM
  YouTube



- Modules in Node: CommonJS & ESM
  YouTube

Hand-picked references for this lesson.